Blatt 03: Git-Quest & LSF-Contact (Git Branches, Methoden-Referenzen, Logging)

Unterlagen

- PDF-Version

A03.1: Git-Spiel

Betrachten Sie erneut die Vorgaben zur "Git-Quest". Die Geschichte des Helden Markus findet im `master`-Branch kein Ende, sondern erst im Hilfsbranch `end`.

Machen Sie nun verschiedene Experimente mit Branches in Git, und starten Sie dabei jeweils mit einem frischen Klon der Vorgaben.

1. Ändern Sie eine Datei, die im Branch `end` nicht verändert wurde. Erzeugen Sie mit diesen Änderungen auf dem `master` einen neuen Commit. Mergen Sie danach den Branch `end` in den `master`-Branch.
2. Ändern Sie nun eine Datei, die auch im Branch `end` verändert wurde. Achten Sie dabei darauf, die Änderung an einer anderen Stelle in der Datei vorzunehmen. Erzeugen Sie mit diesen Änderungen auf dem `master` einen neuen Commit. Mergen Sie danach den Branch `end` in den `master`-Branch.
3. Wie (2), aber ändern Sie nun eine Stelle, die auch im Branch `end` verändert wurde. Erzeugen Sie mit diesen Änderungen auf dem `master` einen neuen Commit. Mergen Sie danach den Branch `end` in den `master`-Branch. Was passiert, wenn die Änderung im `master` identisch zu der in `end` ist? Was passiert, wenn die Änderung im `master` anders ist als in `end`?
4. Wie (2), aber setzen Sie bitte den Branch `end` auf die Spitze von `master`, bevor Sie `end` in `master` mergen.

Was beobachten Sie jeweils? Erklären Sie Ihre Beobachtungen. Wenn es Konflikte gibt: Wie lösen Sie diese auf? Demonstrieren Sie das Vorgehen im Praktikum live.

LSF-Contact

Betrachten Sie die Vorgaben "LSF-Contact". Klonen Sie das Repo und laden Sie das Projekt als Gradle-Projekt in Ihre IDE.

A03.2: Methoden-Referenzen

Sie finden im Package `lsfcontact` eine Klasse `Student`. Jede Instanz dieser Klasse hat mindestens einen Namen (`String`), und man kann verschiedene Konktaktmöglichkeiten per Setter setzen: EMail-Adresse, Telefonnummer, Post-Adresse (alle `String`).

Die Klasse `LsfContactUtil` soll ein Hilfsmodul im LSF simulieren, mit der man die Studierenden kontaktieren kann. Es gibt drei verschiedene Methoden, die jeweils mit einer Liste mit `Student`-Objekten aufgerufen werden und die alle Studierenden mit der entsprechend gesetzten Kontaktoption über diesen Kontaktweg ansprechen. *Beispiel*: Die Methode `emailStudents` filtert alle Studierenden, deren EMail-Adresse gesetzt ist (d.h. deren EMail-Adresse ein nicht-leerer String ist) und "schickt" diesen Studierenden eine "EMail" über den Aufruf der privaten Hilfsmethode `email`.

Die Klasse `Main` erzeugt einige `Student`-Objekte, gruppiert sie in einer Liste und demonstriert die Aufrufe der Methoden in `LsfContactUtil`.

Es fällt auf, dass die drei Methoden `emailStudents`, `phoneStudents` und `writeStudents` algorithmisch identisch sind und sich nur in der Abfrage der entsprechenden Kontaktoption und dem Aufruf der internen Kontakt-Methode unterscheiden. Auch die internen Kontakt-Methoden `email`, `phone` und `write` sind recht einfallslose Code-Duplikate.

Schreiben Sie die Klasse `LsfContactUtil` so um, dass es nur noch eine **`public`** Methode für das Kontaktieren einer Liste von `Student`-Objekten gibt. Fassen Sie ebenfalls die drei **`private`** Hilfsmethoden zu einer neuen Hilfsmethode zusammen - dabei soll es inhaltlich bei dem `System.out.println()` mit den aktuell verwendeten Informationen bleiben. Überlegen Sie, wie Sie die Abfrage der Kontaktmöglichkeit und auch die Kriterien für die Prüfung der Strings von außen als Parameter in die Methode hineingeben können. Passen Sie die Schnittstellen an, so dass der neuen **`public`** Methode zusätzlich zur `List<Student>` passende Methodenreferenzen übergeben werden können. Ändern Sie die Demo-Aufrufe in `Main` entsprechend. Die Klasse `Student` verändern Sie bitte nicht.

Tipp: Gehen Sie schrittweise vor und starten zunächst mit geeigneten Lambda-Ausdrücken. Schaffen Sie es, diese durch Methodenreferenzen zu ersetzen?

Achten Sie darauf, alle Schritte nachvollziehbar in Ihrer Arbeitskopie per Git Commit festzuhalten. Demonstrieren Sie dies im Praktikum.

A03.3: Logging

Bauen Sie für das `LsfContactUtil` ein Logging auf der Basis von `java.util.logging` ein: Jede Benachrichtigung von Studierenden soll in ein gemeinsames CSV-File geloggt werden. Dabei soll pro Logging-Vorgang eine neue Zeile mit den folgenden Informationen angehängt werden:

- Log-Level,
- Name der den Log-Vorgang auslösenden Methode,
- Name der Klasse, in der die den Log-Vorgang auslösenden Methode angesiedelt ist,
- Log-Meldung, bestehend aus
  - Name des kontaktierten Studierenden,
  - genutzte Adresse des Studierenden (Mail- oder Postadresse oder Telefonnummer),
  - Kontaktmodus (wird eine Mail geschickt oder ein Brief geschrieben oder ein Anruf getätigt).

Demonstrieren Sie in der Abgabe, wie Sie im Test oder im Hauptprogramm den Logger steuern können, beispielsweise Änderung der Log-Level oder Abschalten des Loggings.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** ca0ea53 (PDF: replace raw github link w/ 'normal' link, 2025-04-09)